

Part 5 — Detailed Investigation

The following findings were selected because they represent different types of code-quality problems. They affect readability, maintainability, reliability, testability, and code design.

1. Syntax Error

A. Tool finding

- **File:** `eduplatform/eduplatform/models/admin.py`
- **Line:** 199
- **Pylint message:** Unterminated string literal (detected at line 199)
- **Message ID:** `E0001 (syntax-error)`

Pylint reported this error directly in the file.

B. Relevant original code

```
for item in data:

    profile = item.get_profile()

    values = ', '.join(

        line 199:    f"'{str(v).replace("'",
'''')}'" if v is not None else 'NULL'

        for v in profile.values()

    )

    inserts.append(f"INSERT INTO
{table_name} ({columns}) VALUES ({values});")
```

C. Explain the problem

A string was started but was not properly closed with a quotation mark. Because of this, Python cannot correctly understand the code.

This is more serious than a normal style warning because the affected file may not be imported or executed correctly.

D. Quality impact

- **Reliability:** The program may fail before running.
 - **Maintainability:** Developers cannot safely modify or use the affected file until the syntax is fixed.
 - **Testability:** Tests involving this module cannot run correctly.
 - **Correctness:** The program cannot be considered correct if its code cannot be parsed.
-

2. Too Many Branches

A. Tool finding

- **File:** `eduplatform/eduplatform/cli/main.py`
- **Line:** 124
- **Pylint message:** Too many branches (16/12)
- **Message ID:** `R0912 (too-many-branches)`

Pylint found 16 branches where the configured limit was 12.

B. Relevant original code

```
def do_register(self, arg):  
  
    """Register a new user.  
  
    Usage: register <role> "<full_name>"  
<email> <password> [phone] "[address]"  
  
    Examples:
```

```
        register admin "John Doe"
john@example.com mypassword

        register teacher "Jane Smith"
jane@example.com mypassword 1234567890 "123
Main St"
```

```
Roles: student, teacher, parent, admin
```

```
"""
```

```
    print("\n[CLI] Starting user
registration process")
```

```
try:
```

```
    import shlex
```

```
    args = shlex.split(arg)
```

```
    if len(args) < 4:
```

```
        print("Error: Missing required
arguments. Usage: register <role>
\"<full_name>\" <email> <password> [phone]
[address]")
```

```
        print("Note: Enclose names and
addresses with spaces in quotes")

        return

    role_map = {

        'student': Student,

        'teacher': Teacher,

        'parent': Parent,

        'admin': Admin

    }

    role = args[0].lower()

    print(f"[CLI] Registering user with
role: {role}")

    if role not in role_map:

        print(f"Error: Invalid role.
Must be one of: {'', ' '.join(role_map.keys())}")
```

```
        return

    # Extract arguments with proper
handling of optional fields

    full_name = args[1]

    email = args[2]

    password = args[3]

    # Handle optional phone and address

    phone = None

    address = None

    if len(args) > 4:

        # If there's a 5th argument,
it's either a phone number or address

        if len(args) == 5:

            # If it's just one more
argument, treat it as phone if it's all digits
```

```
        if args[4].isdigit():
            phone = args[4]

        else:
            address = args[4]

    else:
        # If there are 6+ arguments,
        # 4th is phone, rest is address

        phone = args[4]

        address = ' '.join(args[5:])

    print(f"[CLI] Registration details -
    Name: {full_name}, Email: {email}")

    if phone:
        print(f"[CLI] Phone: {phone}")

    if address:
        print(f"[CLI] Address:
        {address}")
```

```
        # Create a dictionary of kwargs to
pass to register_user

        user_kwargs = {

            'user_type': role_map[role],

            'full_name': full_name,

            'email': email,

            'password': password

        }

        # Add optional fields if provided

        if phone:

            user_kwargs['phone'] = phone

        if address:

            user_kwargs['address'] = address

        print("[CLI] Calling
auth_service.register_user")
```

```
        # Call register_user with the
prepared arguments

        result =
self.auth_service.register_user(**user_kwargs)

        print(f"[CLI]
auth_service.register_user returned:
{type(result)}")

        if not result:

            print("Error: Registration
failed - No result returned")

            return

        if not isinstance(result, dict):

            print(f"Error: Expected dict
result, got {type(result).__name__}")

            return

        if 'user' not in result:
```

```
        print("Error: Registration
failed - No user in result")

        print(f"Result keys:
{result.keys() if hasattr(result, 'keys') else
'Not a dictionary'}")

        return

    print("[CLI] Registration
successful, processing result...")

    self.current_user = result['user']

    self.current_user_type =
result.get('user_type', 'user')

    self.auth_token =
result.get('token')

    print(f"\n[CLI] Successfully
registered and logged in as
{self.current_user._full_name}
({self.current_user_type})")
```

```
print(f"Welcome to EduPlatform!")

except ValueError as e:

    print(f"Error: {str(e)}")

except Exception as e:

    print(f"\n[CLI] An unexpected error
occurred during registration:")

    print(f"Type: {type(e).__name__}")

    print(f"Error: {str(e)}")

    print("\nPlease check the input and
try again.")
```

C. Explain the problem

The function contains too many different decision paths. This usually means one function is handling too many different situations.

For example, a function with many **if**, **elif**, and other decision statements becomes harder to follow because the programmer has to consider many possible paths.

D. Quality impact

- **Readability:** The function is harder to understand.
- **Maintainability:** Changes become more difficult because many paths must be considered.
- **Testability:** More test cases are needed to cover the different branches.

- **Reliability:** A change in one branch can accidentally affect another branch.

3. Too Many Instance Attributes

A. Tool finding

- **File:** eduplatform/eduplatform/models/assignment.py
- **Line:** 19
- **Pylint message:** Too many instance attributes (14/7)
- **Message ID:** R0902 (too-many-instance-attributes)

The class has 14 instance attributes while the configured limit is 7.

B. Relevant original code

```
class Assignment:

    """Class representing an assignment in the
    educational platform."""

    def __init__(self,

                title: str,

                description: str,

                subject: str,

                teacher_id: str,

                class_id: str,

                due_date: Optional[datetime] =
None,
```

```
        max_points: float = 100.0,

        difficulty:

AssignmentDifficulty =
AssignmentDifficulty.MEDIUM) :
```

C. Explain the problem

The class stores a large number of different pieces of information. Having 14 attributes suggests that the class may be responsible for too many things.

A smaller class with closely related data is generally easier to understand and modify.

D. Quality impact

- **Maintainability:** The class becomes harder to manage.
 - **Readability:** It takes more time to understand what the class represents.
 - **Modifiability:** Changing one part may require checking many attributes.
 - **Testability:** More attributes mean more combinations that may need testing.
-

4. Too Many Arguments

A. Tool finding

- **File:** `eduplatform/eduplatform/models/assignment.py`
- **Line:** 22
- **Pylint message:** Too many arguments (9/5)
- **Message ID:** `R0913 (too-many-arguments)`

Pylint reported 9 arguments where the limit is 5.

B. Relevant original code

```
class Assignment:

    """Class representing an assignment in the
    educational platform."""
```

```
def __init__(self,
               title: str,
               description: str,
               subject: str,
               teacher_id: str,
               class_id: str,
               due_date: Optional[datetime] =
None,
               max_points: float = 100.0,
               difficulty:
AssignmentDifficulty =
AssignmentDifficulty.MEDIUM):
    """Initialize a new assignment.

    Args:
        title: Assignment title
        description: Detailed description
```

```
        subject: Subject this assignment is
for

        teacher_id: ID of the teacher who
created the assignment

        class_id: ID of the class this
assignment is for

        due_date: Optional due date
(defaults to 7 days from creation)

        max_points: Maximum points possible

        difficulty: Assignment difficulty
level

    """

    self._id =
f"assgn_{len(str(hash(str(datetime.now())))))[-6
:]}]"

    self._title = title

    self._description = description

    self._subject = subject

    self._teacher_id = teacher_id

    self._class_id = class_id
```

```

        self._created_at = datetime.now()

        self._due_date = due_date if due_date
else (datetime.now() + timedelta(days=7))

        self._max_points =
max(float(max_points))

        self._difficulty = difficulty

        self._status =
AssignmentStatus.DRAFT.value

        self._submissions: Dict[str, Dict] = {}
# {student_id: submission_data}

        self._grades: Dict[str, Dict] = {} #
{student_id: grade_data}

        self._attachments: List[Dict] = [] #
List of file attachments

```

C. Explain the problem

The function requires too much information through separate parameters. This makes it harder to remember what each argument means and increases the chance of passing the wrong value.

Some of these values may belong together in a class or data object.

D. Quality impact

- **Readability:** Function calls become difficult to understand.
- **Maintainability:** Changing the function requires checking many calls.
- **Testability:** Many combinations of arguments may need to be tested.

- **Reliability:** There is a greater chance of passing arguments in the wrong order.
-

5. Unsubscriptable Object

A. Tool finding

- **File:** `eduplatform/eduplatform/models/assignment.py`
- **Line:** 43
- **Pylint message:** Value `len(str(hash(str(datetime.now()))))` is unsubscriptable
- **Message ID:** `E1136 (unsubscriptable-object)`

This exact expression is shown in the Pylint output.

B. Relevant code fragment reported by Pylint

```
self._id =  
f"assgn_{len(str(hash(str(datetime.now())))))[-6  
:]}"
```

C. Explain the problem

Pylint believes the value is being used in a way that may require indexing or subscripting even though its inferred type does not support that operation.

This indicates that the expression is confusing enough for static analysis to detect a possible type-related problem.

D. Quality impact

- **Reliability:** Incorrect type usage can cause errors when the program runs.
 - **Maintainability:** Complex expressions are harder to understand.
 - **Readability:** The expression is difficult to understand at first glance.
 - **Testability:** Type-related problems may only appear for particular execution paths.
-

6. Broad Exception Handling

A. Tool finding

- **File:** eduplatform/eduplatform/utils/export_utils.py
- **Line:** 296
- **Pylint message:** Catching too general exception `Exception`
- **Message ID:** W0703 (`broad-except`)

Pylint reported this in `export_utils.py`.

B. Relevant original code

```
except Exception as e:  
  
    print(f"Error: {str(e)}")
```

C. Explain the problem

The code catches the general `Exception` type instead of catching a specific error.

This can hide different types of errors. For example, a programming error and a normal file error could both be caught by the same block, making debugging harder.

D. Quality impact

- **Reliability:** Real errors may be hidden.
 - **Maintainability:** Debugging becomes more difficult.
 - **Testability:** Tests may not reveal the actual source of failures.
 - **Error handling:** The program cannot react differently to different error types.
-

7. Bare Except

A. Tool finding

- **File:** eduplatform/eduplatform/utils/export_utils.py
- **Line:** 88
- **Pylint message:** No exception type(s) specified
- **Message ID:** W0702 (`bare-except`)

Pylint reported a bare `except` at line 88.

B. Relevant original code

```
except Exception as e:

    raise IOError(f"Failed to export to
Excel: {str(e)}")
```

C. Explain the problem

A bare `except:` catches almost any exception. This makes it difficult to know which errors the programmer expected.

It can also hide unexpected problems instead of allowing them to be fixed.

D. Quality impact

- **Reliability:** Unexpected errors may be hidden.
 - **Maintainability:** Debugging becomes harder.
 - **Testability:** Some failures may be silently swallowed.
 - **Error handling:** The program loses control over different types of errors.
-

8. Protected Member Access

A. Tool finding

- **File:** `eduplatform/eduplatform/repositories/schedule_repository.py`
- **Line:** 11
- **Pylint message:** Access to a protected member `_id` of a client class
- **Message ID:** `W0212 (protected-access)`

Pylint reported many similar protected-member accesses in this repository.

B. Relevant code fragment

```
def _get_key(self, item: Schedule) -> str:

    """Get the unique key for a schedule
(its ID)."""
```

```
return item._id
```

C. Explain the problem

Names beginning with `_` are normally treated as internal or protected members of a class.

Accessing them directly from another class breaks the intended separation between the class's internal data and the code using that class.

A public method or property can be used instead.

D. Quality impact

- **Maintainability:** Changes inside the class can break other code.
 - **Modifiability:** Internal implementation becomes harder to change safely.
 - **Encapsulation:** The class's internal data is exposed to other parts of the program.
 - **Reliability:** Direct access can allow data to be changed incorrectly.
-

9. Unused Variable

A. Tool finding

- **File:** `eduplatform/eduplatform/services/assignment_service.py`
- **Line:** 96
- **Pylint message:** Unused variable `notification`
- **Message ID:** `W0612 (unused-variable)`

Pylint specifically reports that `notification` is assigned but not used.

B. Relevant original code

```
def _notify_students(self, assignment:
Assignment) -> None:
    """Notify students about a new
assignment."""
```

```
        # In a real implementation, we would get
the list of students in the class

        # For this example, we'll just create a
notification for the assignment

        notification =
self.notification_repo.create_notification(

recipient_id=f"class_{assignment._class_id}",
# This would be expanded to individual student
IDs

        title=f"New Assignment:
{assignment._title}",
        message=f"A new assignment has been
posted for {assignment._subject}. Due:
{assignment._due_date.strftime('%b %d, %Y')}",

notification_type=NotificationType.ASSIGNMENT.v
alue,

        priority=NotificationPriority.HIGH,
        related_entity_id=assignment._id,
        related_entity_type='assignment',
        metadata={
            'assignment_id': assignment._id,
            'due_date':
assignment._due_date.isoformat(),
```

```

        'subject': assignment._subject,
        'max_points':
assignment._max_points
    }
)

```

C. Explain the problem

The program creates or stores a value in `notification`, but the value is not used afterward.

This may mean the variable is unnecessary, or it may indicate that the programmer forgot to use the result.

D. Quality impact

- **Readability:** Unused variables make the code more confusing.
- **Maintainability:** Developers have to spend time deciding whether the variable is needed.
- **Reliability:** If the result was supposed to be used, an important operation may be missing.
- **Code cleanliness:** Dead code makes the program unnecessarily complicated.

10. Line Too Long

A. Tool finding

- **File:** `eduplatform/eduplatform/services/assignment_service.py`
- **Line:** 99
- **Pylint message:** Line too long (133/100)
- **Message ID:** `C0301 (line-too-long)`

Pylint reports that line 99 contains 133 characters while the limit is 100.

B. Relevant original code

```

        message=f"A new assignment has been
posted for {assignment._subject}. Due:
{assignment._due_date.strftime('%b %d, %Y')}",

```

C. Explain the problem

The line contains too much code or text on one line. Long lines are harder to read because the reader may need to scroll horizontally or mentally break the statement into smaller parts.

D. Quality impact

- **Readability:** The code is harder to read.
- **Maintainability:** Understanding and modifying the line takes more effort.
- **Code quality:** Consistent formatting is reduced.
- **Team development:** Long lines make code reviews more difficult.